

A STRUCTURED REFERENCE · INTERVIEW PREPARATION

DSA Patterns Mastery Guide

Twenty-one recurring patterns behind almost every technical-interview coding question — explained with analogies, worked theory, and verified solutions in Python and C++.

21

PATTERNS

7

PARTS

2

LANGUAGES

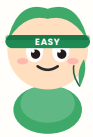
100%

CODE
VERIFIED

EVERY SOLUTION IN THIS GUIDE WAS COMPILED / EXECUTED AGAINST TEST CASES BEFORE PUBLICATION

The Difficulty & Frequency System

Every chapter opens with a character badge and a signal-bar meter. The character's costume tells you how conceptually demanding the pattern is; the bars tell you how often it actually shows up in real interviews. Neither number is about the code length — a "Boss Level" pattern can still be twenty lines. It's about how much judgment the pattern demands.



FOUNDATION

Simple mechanics, huge return on investment. Master these first.



CORE SKILL

The backbone of most medium-difficulty interview questions.



ADVANCED

Needs a real invariant / proof, not just a template.



BOSS LEVEL

Where most candidates stall. Worth the extra rep count.

■ INTERVIEW FREQUENCY METER



Low — good to know, rarely the whole interview



Medium — shows up regularly, especially at larger companies



High — a genuine staple of FAANG-style loops



Very High — expect it, in some form, almost every loop

Table of Contents

PART I Pointer & Window Patterns

01	Two Pointers	FOUNDATION
02	Sliding Window	FOUNDATION
03	Fast & Slow Pointers	FOUNDATION
04	Cyclic Sort	FOUNDATION
05	Merge Intervals	CORE SKILL
06	In-place Reversal of a Linked List	CORE SKILL

PART II Tree & Heap Patterns

07	Tree BFS (Level Order)	FOUNDATION
08	Tree DFS (Pre/In/Post-order)	FOUNDATION
09	Two Heaps	ADVANCED
10	Top K Elements	CORE SKILL
11	K-way Merge	CORE SKILL

PART III Search & Backtracking

12	Modified Binary Search	CORE SKILL
13	Subsets / Backtracking	CORE SKILL

PART IV Graph Patterns

14	Topological Sort	ADVANCED
15	Union-Find (Disjoint Set)	ADVANCED

PART V Dynamic Programming

16	0/1 Knapsack	BOSS LEVEL
17	Unbounded Knapsack & String DP	BOSS LEVEL

PART VI

Strings & Stacks

18	Trie (Prefix Tree)	CORE SKILL
19	Monotonic Stack	CORE SKILL

PART VII

Optimization Toolkit

20	Greedy Algorithms	ADVANCED
21	Bit Manipulation	ADVANCED



PART I

Pointer & Window Patterns

Six techniques built on the same idea: instead of re-scanning data from scratch, keep one or more markers moving through it with a clear invariant about what's already been ruled out. This is the cheapest complexity win in the entire interview canon — most of these turn an $O(n^2)$ brute force into $O(n)$.

01 · Two Pointers

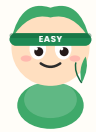
02 · Sliding Window

03 · Fast & Slow Pointers

04 · Cyclic Sort

05 · Merge Intervals

06 · In-place Reversal of a Linked List



Two Pointers

Walk toward the answer from both ends at once.

THE ANALOGY

Two Friends on a Rope

Picture two friends standing at opposite ends of a long rope stretched across a sorted number line. They each start walking toward each other. Every time they check the distance between them against a target, they decide who should step inward: if they're too far apart, the friend on the 'too small' side moves; if they overshoot, the friend on the 'too big' side steps back. They never backtrack, and they're guaranteed to meet in the middle — because the rope is already sorted, so every step inward eliminates a whole slice of impossibilities instead of just one element.

THE THEORY

Two Pointers works whenever the input is sorted, or can be treated as ordered along some axis, and you want to examine pairs, triplets, or a subset defined by a relationship between values at two positions. Instead of the brute-force $O(n^2)$ approach of checking every pair, you set one pointer at each end (or one trailing the other) and move them based on a comparison, discarding a whole partition of impossible answers with every step. The invariant that makes this safe: everything already 'passed over' by a pointer has been provably ruled out for the current search.

- Place `left` at index 0 and `right` at the last index (same-direction pointers form a different sub-family).
- Compute the relationship the problem cares about (sum, difference, product) using `arr[left]` and `arr[right]`.
- Compare that relationship to the target and move exactly one pointer inward — whichever one can fix the imbalance.
- Stop when the pointers cross, or the moment you find the answer.

TIME COMPLEXITY

$O(n)$ — each pointer visits every index at most once

SPACE COMPLEXITY

$O(1)$ extra space

WORKED EXAMPLE

Pair with Target Sum

Given a sorted array of integers and a target value, return the indices of the two numbers that add up exactly to the target. Assume exactly one valid pair exists.

PYTHON

PAIR WITH TARGET SUM

```
def pair_with_target_sum(arr, target):
    left, right = 0, len(arr) - 1
    while left < right:
        current = arr[left] + arr[right]
        if current == target:
            return [left, right]
        if current < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]

# Example
print(pair_with_target_sum([2, 3, 5, 8, 12], 15)) # [1, 4] (3 + 12)
```

C++

PAIR WITH TARGET SUM

```
#include <vector>
using namespace std;

vector<int> pairWithTargetSum(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left < right) {
        int current = arr[left] + arr[right];
        if (current == target) return {left, right};
        if (current < target) left++;
        else right--;
    }
    return {-1, -1};
}

// Example
// vector<int> arr = {2, 3, 5, 8, 12};
// pairWithTargetSum(arr, 15) -> {1, 4} (3 + 12)
```

COMMON PITFALL

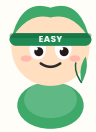
The classic mistake is moving both pointers on every iteration ‘to be safe.’ That silently skips valid pairs — move only the pointer that logically must move, one step, one side, one decision at a time.

PRO TIP

If the array isn't sorted but the problem still smells like Two Pointers (3Sum, for instance), sort it first — the $O(n \log n)$ sort is cheaper than the $O(n^2)/O(n^3)$ brute force it replaces.



Sliding Window



FOUNDATION



Grow and shrink a moving frame instead of re-scanning from scratch.

THE ANALOGY

A Camera Viewfinder on a Reel of Film

Imagine dragging a camera viewfinder across a long filmstrip. Instead of lifting the camera and re-aiming from the start for every possible frame, you slide the right edge forward to include new footage, and when the shot breaks your rule (too many repeats, sum too big – whatever the constraint is) you slide the left edge forward to crop the badness out. You never re-examine footage twice – the frame only ever grows or shrinks at its edges, one splice at a time.

THE THEORY

Sliding Window fixes ‘find the best contiguous subarray/substring satisfying some condition,’ a family that looks like it needs checking every one of the $O(n^2)$ sub-ranges. Because the window only expands right and contracts left, each index enters the window exactly once and leaves exactly once, so the scan is $O(n)$ instead of $O(n^2)$. The trick is maintaining a running summary of ‘what’s currently inside the window’ (a sum, a frequency map, a distinct-character count) so you never recompute it from scratch after a shrink.

- Expand right one step at a time, folding the new element into your running window state.
- While the window violates the constraint, shrink from left, removing that element from the running state.
- Whenever the window is valid, update your answer (max length, min length, count, etc.).
- Repeat until right reaches the end of the array or string.

TIME COMPLEXITY

$O(n)$ – right and left each traverse the array once

SPACE COMPLEXITY

$O(k)$ for the window's tracking structure (k = alphabet size or window contents)

WORKED EXAMPLE

Longest Substring Without Repeating Characters

Given a string, find the length of the longest contiguous substring that contains no repeated characters.

PYTHON

LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS


```
def length_of_longest_substring(s):
    char_index = {}
    left = 0
    max_len = 0
    for right, ch in enumerate(s):
        if ch in char_index and char_index[ch] >= left:
            left = char_index[ch] + 1
        char_index[ch] = right
        max_len = max(max_len, right - left + 1)
    return max_len

# Example
print(length_of_longest_substring("abcabcbb")) # 3 ("abc")
```

C++

LONGEST SUBSTRING WITHOUT REPEATING CHARACTERS

```
#include <string>
#include <unordered_map>
#include <algorithm>
using namespace std;

int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> charIndex;
    int left = 0, maxLen = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        char ch = s[right];
        if (charIndex.count(ch) && charIndex[ch] >= left) {
            left = charIndex[ch] + 1;
        }
        charIndex[ch] = right;
        maxLen = max(maxLen, right - left + 1);
    }
    return maxLen;
}

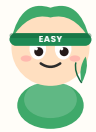
// Example
// lengthOfLongestSubstring("abcabcbb") -> 3 ("abc")
```

COMMON PITFALL

Forgetting to check that a repeated character's stored 'last seen index' is still inside the current window. If it's from before left, it's stale — moving left because of it shrinks the window incorrectly and undercounts the answer.

PRO TIP

Ask: does the window's validity change monotonically as I move an edge? If yes, sliding window applies. If the condition can flip back and forth unpredictably, you likely need a different pattern.



Fast & Slow Pointers

A tortoise and a hare always meet inside a loop.



THE ANALOGY

The Tortoise and the Hare on a Circular Track

Two runners start at the same point, but the hare always moves twice as fast as the tortoise. If the track is a straight line with an end, the hare simply finishes first — nothing interesting happens. But if the track secretly loops back on itself, the hare eventually laps the tortoise from behind, and the instant they occupy the same spot again, you've proven a loop exists. A second, gentler lap from the start then pinpoints exactly where the loop begins.

THE THEORY

Fast & Slow Pointers (Floyd's Cycle Detection) answers 'does this linked structure loop, and where' without any extra memory for visited nodes. Two pointers walk the same list, one step at a time for the slow pointer, two steps for the fast one. With no cycle, the fast pointer simply reaches null first. With a cycle, the gap between them shrinks by exactly one node every step once both are inside the loop, guaranteeing a meeting. A clean mathematical fact — distance from head to cycle-start equals distance from the meeting point to cycle-start, at equal speed — finds the exact start node in one more pass.

- Start both `slow` and `fast` at the head.
- Advance `slow` by 1 and `fast` by 2 on every iteration.
- If `fast` (or `fast.next`) hits null, there's no cycle — stop.
- If `slow` and `fast` ever meet, reset one pointer to head and advance both one step at a time until they meet again — that node is the cycle's start.

TIME COMPLEXITY

$O(n)$ — bounded by the number of nodes before the pointers meet

SPACE COMPLEXITY

$O(1)$ — only two pointers, no visited-set needed

WORKED EXAMPLE

Linked List Cycle II

Given the head of a linked list that may contain a cycle, return the node where the cycle begins, or null if there is no cycle.

PYTHON

LINKED LIST CYCLE II

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def detect_cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            pointer = head
            while pointer != slow:
                pointer = pointer.next
                slow = slow.next
            return pointer
    return None
```

C++

LINKED LIST CYCLE II

```
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* detectCycleStart(ListNode* head) {
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            ListNode* pointer = head;
            while (pointer != slow) {
                pointer = pointer->next;
                slow = slow->next;
            }
            return pointer;
        }
    }
    return nullptr;
}
```

COMMON PITFALL

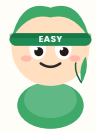
Checking `fast.next.next` without first checking `fast.next` for null crashes on odd-length non-cyclic lists. Always guard both `fast` and `fast.next` before advancing.

PRO TIP

The 'second pointer from head, same speed' trick isn't a coincidence — it's provable with modular arithmetic on the distances. Memorize the two-phase structure, not just the code.



Cyclic Sort



FOUNDATION

When numbers are their own array indices, sort by teleporting them home.



THE ANALOGY

Assigned Seating at a Banquet

Imagine a banquet hall where seat number 3 should hold the guest wearing badge '3.' You walk down the row of seats. If the person in the current seat has the wrong badge, you don't just note it — you swap them with whoever is sitting in the seat matching their badge. You only advance once the current seat is correctly occupied (or permanently unfixable). By the end, almost every badge sits in its rightful seat, and any seat still wrong instantly reveals which badge is missing or duplicated.

THE THEORY

Cyclic Sort is a narrow but powerful trick for arrays containing numbers from a known, dense range (typically $0..n-1$ or $1..n$), used to find a missing, duplicate, or misplaced value. Rather than a general-purpose sort, you exploit the fact that a correctly placed value at index i should equal i (or $i+1$). You repeatedly swap the current element into its 'home' index until the value at the current index is already correct, then move forward. Every swap places at least one number permanently, so the whole array is fixed in linear time.

- At index i , look at the value $v = \text{arr}[i]$.
- If v belongs at a different index ($v < n$ and $\text{arr}[v] \neq v$), swap $\text{arr}[i]$ and $\text{arr}[v]$.
- If v is already home or out of range, move i forward.
- After one pass, scan again: any index where $\text{arr}[i] \neq i$ reveals the missing/duplicate number.

TIME COMPLEXITY

$O(n)$ — each swap permanently places one element, so at most n swaps happen in total

SPACE COMPLEXITY

$O(1)$ — sorted in place

WORKED EXAMPLE

Find the Missing Number

Given an array containing n distinct numbers taken from the range 0 to n , find the one number missing from the range.

PYTHON

FIND THE MISSING NUMBER

```
def find_missing_number(nums):
    i = 0
    n = len(nums)
    while i < n:
        j = nums[i]
        if j < n and nums[i] != nums[j]:
            nums[i], nums[j] = nums[j], nums[i]
        else:
            i += 1
    for i in range(n):
        if nums[i] != i:
            return i
    return n
```

C++

FIND THE MISSING NUMBER

```
#include <vector>
using namespace std;

int findMissingNumber(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int j = nums[i];
        if (j < n && nums[i] != nums[j]) {
            swap(nums[i], nums[j]);
        } else {
            i++;
        }
    }
    for (i = 0; i < n; i++) {
        if (nums[i] != i) return i;
    }
    return n;
}
```

COMMON PITFALL

Using an `if` instead of a loop-friendly re-check at the same index causes some elements to never reach their home position. The placement step must keep swapping at index `i` until it's genuinely correct before advancing.

PRO TIP

Whenever a problem promises 'numbers from 1 to `n`' or '0 to `n`' with possible duplicates or one missing/duplicate value, Cyclic Sort is almost always faster and leaner than a hash set.



Merge Intervals



CORE SKILL



Sort by start time, then glue anything that overlaps.

THE ANALOGY

Overlapping Shifts on a Work Calendar

You're consolidating a messy shift calendar. First, line every shift up by start time. Then walk down the list holding the 'currently open' shift. If the next shift starts before your open one ends, they overlap — stretch the open shift's end to cover whichever finishes later and keep walking. The moment a shift starts after your open one has already ended, close it out, write it down, and open the new one. By the end, every touching or overlapping run of shifts has collapsed into one clean block.

THE THEORY

Merge Intervals handles any problem about ranges that touch or overlap — meeting rooms, calendar events, numeric ranges. Once intervals are sorted by start time, you never need to compare an interval against anything except the most recently merged one: if interval B starts before or exactly when the last merged interval ends, they belong together, so extend the end to the max of the two. Otherwise B starts a fresh block. This turns an apparently pairwise, $O(n^2)$ -shaped problem into one linear pass after the sort.

- Sort all intervals by their start value.
- Seed the result list with the first interval as the 'currently open' interval.
- For each following interval, compare its start to the open interval's end.
- Overlap → extend the open interval's end to the max of the two ends. No overlap → push the open interval to results and open the new one.

TIME COMPLEXITY

$O(n \log n)$ — dominated by the sort; the merge pass itself is $O(n)$

SPACE COMPLEXITY

$O(n)$ for the output list

WORKED EXAMPLE

Merge Intervals

Given an array of intervals, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input.

PYTHON

MERGE INTERVALS

```
def merge_intervals(intervals):
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:
            last[1] = max(last[1], current[1])
        else:
            merged.append(current)
    return merged
```

C++

MERGE INTERVALS

```
#include <vector>
#include <algorithm>
using namespace std;

vector<vector<int>>> mergeIntervals(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};
    sort(intervals.begin(), intervals.end(),
        [](const vector<int>& a, const vector<int>& b) { return a[0] < b[0]; });
    vector<vector<int>>> merged;
    merged.push_back(intervals[0]);
    for (size_t i = 1; i < intervals.size(); i++) {
        auto& last = merged.back();
        if (intervals[i][0] <= last[1]) {
            last[1] = max(last[1], intervals[i][1]);
        } else {
            merged.push_back(intervals[i]);
        }
    }
    return merged;
}
```

COMMON PITFALL

Comparing the new interval's start to the *previous original* interval's end instead of the *currently merged* interval's end. After two merges, the open interval's true end may be far larger than any single original interval's end — always compare against the running merged value.

PRO TIP

'Insert Interval' and 'Meeting Rooms' are this same pattern wearing different clothes: the moment you see 'overlapping ranges,' reach for sort-by-start plus a single running interval.



In-place Reversal of a Linked List

Flip pointers with only three hands, never derailing the train.



THE ANALOGY

Flipping Train Cars Mid-Journey

Picture reversing a moving train's direction, but you're only allowed to hold three cars at a time: the one behind, the one you're working on, and the one ahead. You detach the current car from the one ahead, weld it instead to the one behind, then shuffle all three hands forward by one car. Do this down the whole train and it runs backward — without ever needing a second train (extra array or list) to hold a copy.

THE THEORY

In-place Reversal rewires pointer direction using $O(1)$ extra memory by keeping exactly three references — `prev`, `curr`, `next` — and updating one link per step. It generalizes cleanly to 'reverse only a sub-section' or 'reverse every group of k nodes,' which is why interviewers love it: the base operation is simple, but composing it (sub-range reversal, k -group reversal, checking enough nodes remain for a full group) tests whether you understand the mechanics rather than having memorized one specific solution.

- Set `prev = null`, `curr = head`.
- Save `next = curr.next` before overwriting anything.
- Rewire: `curr.next = prev`.
- Shift the window forward: `prev = curr`, `curr = next`. Repeat until `curr` is null (or k nodes are done, for the grouped variant).

TIME COMPLEXITY

$O(n)$ — every node's pointer is rewired exactly once

SPACE COMPLEXITY

$O(1)$ iteratively; $O(n/k)$ recursion stack for the recursive k -group version

WORKED EXAMPLE

Reverse Nodes in k -Group

Given a linked list, reverse the nodes k at a time and return the modified list. If the number of remaining nodes at the end is less than k , leave that final group unchanged.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_k_group(head, k):
    node = head
    count = 0
    while node and count < k:
        node = node.next
        count += 1
    if count < k:
        return head # fewer than k nodes remain, leave as is

    prev = None
    curr = head
    for _ in range(k):
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt

    head.next = reverse_k_group(curr, k) # curr is the (k+1)-th node
    return prev

```

C++

REVERSE NODES IN K-GROUP

```

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* node = head;
    int count = 0;
    while (node && count < k) {
        node = node->next;
        count++;
    }
    if (count < k) return head;

    ListNode* prev = nullptr;
    ListNode* curr = head;
    for (int i = 0; i < k; i++) {
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    head->next = reverseKGroup(curr, k);
    return prev;
}

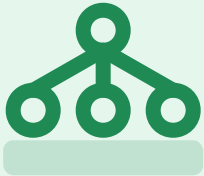
```

COMMON PITFALL

Overwriting `curr.next` before saving it into a temporary variable loses the rest of the list forever — always capture `next` first. This is the single most common linked-list bug.

PRO TIP

Before writing a line of code, draw three boxes — `prev`, `curr`, `next` — and trace two full iterations by hand. If you can't trace it on paper, you can't code it under interview pressure.



PART II

Tree & Heap Patterns

Trees are graphs with no cycles to worry about, which is exactly why they're the friendliest place to learn traversal thinking. Heaps enter the picture the moment a problem needs 'the best of what I've seen so far' on demand, without re-sorting everything from scratch.

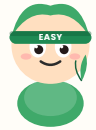
07 · Tree BFS (Level Order)

08 · Tree DFS (Pre/In/Post-order)

09 · Two Heaps

10 · Top K Elements

11 · K-way Merge



Tree BFS (Level Order)

Explore a tree one generation at a time.

THE ANALOGY

Reading a Family Tree Generation by Generation

Imagine presenting a family tree not name-by-name down one branch, but generation by generation: all grandparents first, then every parent, then every grandchild — each row fully announced before moving to the next. You need a waiting line (a queue): as you announce someone, you add their children to the back of the line, guaranteeing you finish everyone at the current depth before looking at the next one down.

THE THEORY

Tree (and graph) BFS visits nodes level by level using a FIFO queue: push the root, then repeatedly pop a node, process it, and push its children. Because a queue preserves arrival order, all nodes at depth d are fully dequeued before any node at depth $d+1$ is even considered. This makes BFS the natural choice whenever the question is inherently about levels or the shortest number of steps — level-order output, zigzag traversal, minimum depth, or shortest path in an unweighted graph.

- Push the root into a queue.
- While the queue isn't empty, record its current size — that's exactly how many nodes belong to this level.
- Pop that many nodes, process each, and push their children.
- Repeat until the queue empties; each full round is one level of the tree.

TIME COMPLEXITY

$O(n)$ — every node is enqueued and dequeued exactly once

SPACE COMPLEXITY

$O(w)$ where w is the widest level of the tree (worst case $O(n)$)

WORKED EXAMPLE

Binary Tree Level Order Traversal

Given the root of a binary tree, return the values of its nodes grouped level by level, from left to right.

PYTHON

BINARY TREE LEVEL ORDER TRAVERSAL

```
from collections import deque
```

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level_size = len(queue)
        level = []
        for _ in range(level_size):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result

```

C++

BINARY TREE LEVEL ORDER TRAVERSAL

```

#include <vector>
#include <queue>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> result;
    if (!root) return result;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int levelSize = q.size();
        vector<int> level;
        for (int i = 0; i < levelSize; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}

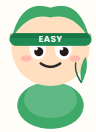
```

COMMON PITFALL

Reading the queue's length *inside* the inner loop instead of capturing it once before the loop starts. Since you're pushing children into the same queue mid-loop, the length changes underneath you and levels bleed into each other.

PRO TIP

Anytime you see 'level,' 'depth,' 'shortest path' (unweighted graph), or 'minimum steps,' your first instinct should be BFS, not DFS — DFS finds a path; BFS finds the shortest one for free.



Tree DFS (Pre/In/Post-order)

Commit to one path fully before you ever backtrack.

THE ANALOGY

Exploring a Maze by Committing to a Corridor

You're exploring a maze with a ball of string. At every junction, you pick one corridor and follow it as far as it goes — past every further junction — before ever considering the corridors you skipped. Only at a dead end do you retrace your steps to the most recent junction with an unexplored corridor. This 'go deep first, backtrack only when stuck' strategy is exactly what the call stack gives you for free in a recursive tree traversal.

THE THEORY

Tree DFS explores as far down one branch as possible before backtracking, using either recursion (the call stack does the bookkeeping) or an explicit stack. The three orderings — preorder (node, then left, then right), inorder (left, node, right), and postorder (left, right, node) — aren't arbitrary; each matches a real use case. Preorder is for copying/serializing a tree, inorder gives sorted output on a binary search tree, and postorder is for anything where children must be fully resolved before the parent (deleting a tree, computing subtree sizes, evaluating an expression tree).

- Define a base case: if the current node is null, return immediately.
- Decide your ordering: process the current node's value before, between, or after recursing into its children.
- Recurse into the left child, then the right child (or vice versa for specific problems).
- For path-based problems, push the value onto a running path before recursing and pop it after — this is what makes backtracking correct.

TIME COMPLEXITY

$O(n)$ — every node visited exactly once

SPACE COMPLEXITY

$O(h)$ recursion stack, h = tree height ($O(\log n)$ balanced, $O(n)$ worst case)

WORKED EXAMPLE

Path Sum II — All Root-to-Leaf Paths

Given the root of a binary tree and a target sum, return every root-to-leaf path where the node values along the path add up exactly to the target.

PYTHON

PATH SUM II — ALL ROOT-TO-LEAF PATHS

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def path_sum(root, target_sum):
    result = []

    def dfs(node, remaining, path):
        if not node:
            return
        path.append(node.val)
        remaining -= node.val
        if not node.left and not node.right and remaining == 0:
            result.append(list(path))
        else:
            dfs(node.left, remaining, path)
            dfs(node.right, remaining, path)
        path.pop()

    dfs(root, target_sum, [])
    return result
```

C++

PATH SUM II — ALL ROOT-TO-LEAF PATHS

```
#include <vector>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

void dfs(TreeNode* node, int remaining, vector<int>& path, vector<vector<int>>& result) {
    if (!node) return;
    path.push_back(node->val);
    remaining -= node->val;
    if (!node->left && !node->right && remaining == 0) {
        result.push_back(path);
    } else {
        dfs(node->left, remaining, path, result);
        dfs(node->right, remaining, path, result);
    }
    path.pop_back();
}

vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
```



```
vector<vector<int>> result;  
vector<int> path;  
dfs(root, targetSum, path, result);  
return result;  
}
```

COMMON PITFALL

Appending the current path to the result *by reference* instead of a copy. Since the same list object keeps mutating as you backtrack, every 'saved' path silently turns into whatever the path looks like by the time recursion finishes — always store a real copy (`list(path)` / `vector<int>(path)`).

PRO TIP

Need the shortest or level-based answer? Reach for BFS. Need all paths, full branch exploration, or a parent computed from child results (tree diameter, subtree sums)? Reach for DFS.



Two Heaps

ADVANCED

Keep the smaller half and the larger half perfectly balanced.



THE ANALOGY

A Perfectly Balanced Seesaw

Picture a seesaw where every number seen so far sits on one side or the other: the smaller half sits on the left arm (you only ever look at the biggest of that group, closest to the pivot), the larger half sits on the right arm (you only ever look at the smallest of that group). When a new number arrives, you place it on the correct side, and if one arm gets more than one heavier than the other, you shift the middlemost value across the pivot. The two values closest to the pivot are always your median.

THE THEORY

Two Heaps solves ‘give me the median (or an order-statistic) of a stream of numbers, continuously, as new numbers arrive.’ Sorting on every insert costs $O(n \log n)$ per number; Two Heaps gets it to $O(\log n)$ per number by maintaining a max-heap for the smaller half of the data and a min-heap for the larger half, kept within one element of each other in size. The median is then either the top of the larger heap (odd total) or the average of both tops (even total) — an $O(1)$ lookup.

- Push the new number onto the max-heap (`small`).
- Immediately pop the max-heap's top and push it onto the min-heap (`large`) — guarantees every value in `small` is $\leq$ every value in `large`.
- If `large` now has more elements than `small`, pop its smallest and push it back into `small` to rebalance.
- Median: top of `small` if sizes are unequal, else the average of both tops.

TIME COMPLEXITY

$O(\log n)$ per insertion; $O(1)$ per median query

SPACE COMPLEXITY

$O(n)$ to hold the full stream across both heaps

WORKED EXAMPLE

Find Median from Data Stream

Design a data structure that supports adding integers one at a time from a stream and, at any point, can efficiently return the median of all elements seen so far.

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = [] # max-heap, stored as negated values
        self.large = [] # min-heap

    def add_num(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0
```

C++

FIND MEDIAN FROM DATA STREAM

```
#include <queue>
#include <vector>
using namespace std;

class MedianFinder {
    priority_queue<int> small; // max-heap
    priority_queue<int, vector<int>, greater<int>> large; // min-heap
public:
    void addNum(int num) {
        small.push(num);
        large.push(small.top());
        small.pop();
        if (large.size() > small.size()) {
            small.push(large.top());
            large.pop();
        }
    }
    double findMedian() {
        if (small.size() > large.size()) return small.top();
        return (small.top() + large.top()) / 2.0;
    }
};
```

COMMON PITFALL

Skipping the ‘push into small, then immediately move the max back to large’ dance and instead comparing the new number directly against the current tops. It looks equivalent for a few test cases but breaks the invariant that every element of `small` is $\leq$ every element of `large`, producing a wrong median a few insertions later.

PRO TIP

‘Running median,’ ‘sliding window median,’ or ‘kth smallest but the data keeps changing’ — Two Heaps almost always beats re-sorting.



Top K Elements

Only k VIP seats — a newcomer must outrank someone to get in.



THE ANALOGY

A Nightclub with Exactly K VIP Seats

A VIP section has exactly k seats, starting full of the first k guests, ranked least to most impressive by the bouncer (a min-heap, so the least impressive VIP sits nearest the door). Every new arrival is compared only to that weakest current VIP. Not more impressive? Instant rejection, $O(1)$. More impressive? The weakest VIP is shown the door and the newcomer takes the seat, and the heap quietly re-sorts to find the new weakest.

THE THEORY

Top K Elements problems (k th largest, k closest points, top k frequent) all share a shape: you don't need a full sort, only a separation of the top k from everyone else. A min-heap of size k does this in $O(n \log k)$ instead of $O(n \log n)$ for a full sort — a meaningful difference when k is small and n is huge. The min-heap's root is always the weakest member of the current top- k group, exactly the piece of information needed to decide whether a new element deserves to join.

- Build a min-heap from the first k elements.
- For every remaining element, compare it to the heap's root (the current smallest of the top- k group).
- If the new element is larger, pop the root and push the new element.
- After processing everything, the heap holds exactly the top- k elements; the root is the k th largest overall.

TIME COMPLEXITY

$O(n \log k)$ — much better than $O(n \log n)$ full sort when $k \ll n$

SPACE COMPLEXITY

$O(k)$ for the heap

WORKED EXAMPLE

K th Largest Element in an Array

Given an unsorted array of integers and an integer k , find the k th largest element without fully sorting the array.

```
import heapq

def find_kth_largest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for num in nums[k:]:
        if num > heap[0]:
            heapq.heapreplace(heap, num)
    return heap[0]
```

C++

KTH LARGEST ELEMENT IN AN ARRAY

```
#include <vector>
#include <queue>
using namespace std;

int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap(nums.begin(), nums.begin() + k);
    for (int i = k; i < (int)nums.size(); i++) {
        if (nums[i] > minHeap.top()) {
            minHeap.pop();
            minHeap.push(nums[i]);
        }
    }
    return minHeap.top();
}
```

COMMON PITFALL

Using a max-heap of the entire array instead of a min-heap capped at size k . It's still correct, but throws away the whole point of the pattern — you're back to $O(n \log n)$ with extra memory.

PRO TIP

'Top k ' or 'kth' in a problem statement is the signal. The only design decision left is min-heap vs. max-heap: cap the heap at size k and use whichever type cheaply exposes the worst member of your current top- k group.



K-way Merge

Merge k sorted lines by always pulling the smallest front.



THE ANALOGY

Merging K Sorted Checkout Lines

You have k separate checkout lines, and unusually, every line is already sorted by ticket number, front to back. You want one master line in sorted order. The obvious move: look at the front person of every line, pull whichever has the lowest ticket number into your master line, and let the next person in that line step up. Repeat. You never need to look further back than the very front of each line, because sortedness guarantees nothing smaller is hiding behind it.

THE THEORY

K-way Merge generalizes the classic two-list merge (from merge sort) to k lists at once. Instead of comparing k front-elements with a linear scan every round ($O(k)$ per element, $O(n \cdot k)$ total), you keep all k 'current fronts' in a min-heap of size k . Popping the minimum and pushing that same list's next element costs $O(\log k)$, so the whole merge runs in $O(n \log k)$ — the same complexity win as Top K Elements, applied to merging instead of selecting.

- Push the first node/element of every list into a min-heap, tagged with which list it came from.
- Pop the minimum, append it to the result.
- Push that popped element's next item from the same source list (if one exists).
- Repeat until the heap is empty — the result is the fully merged, sorted sequence.

TIME COMPLEXITY

$O(n \log k)$ — n total elements, heap operations cost $\log k$

SPACE COMPLEXITY

$O(k)$ for the heap, plus $O(n)$ for the output

WORKED EXAMPLE

Merge k Sorted Linked Lists

You are given an array of k linked lists, each sorted in ascending order. Merge all the lists into one sorted linked list and return it.

```

import heapq

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode(0)
    tail = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next

```

C++

MERGE K SORTED LINKED LISTS

```

#include <vector>
#include <queue>
using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* mergeKLists(vector<ListNode*> lists) {
    auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> heap(cmp);
    for (auto node : lists) {
        if (node) heap.push(node);
    }
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (!heap.empty()) {
        ListNode* node = heap.top(); heap.pop();
        tail->next = node;
        tail = tail->next;
        if (node->next) heap.push(node->next);
    }
    return dummy.next;
}

```

COMMON PITFALL

Forgetting a tie-breaker (like the source list index) when heap values can repeat. Comparing raw nodes/objects directly can misbehave when equal-value entries aren't natively comparable — always compare by value first, source index second.

PRO TIP

Merging or interleaving more than two already-sorted things — lists, streams, files — calls for a heap of size k , not a chain of pairwise two-way merges (which costs more passes over the data).



PART III

Search & Backtracking

Two ways of cutting down a huge search space: Modified Binary Search throws away half the space with one comparison; Backtracking explores the whole decision tree but prunes the instant a branch is provably dead.

12 • Modified Binary Search

13 • Subsets / Backtracking



Modified Binary Search

Even a rotated or bounded array still has a 'sorted half' to search.



THE ANALOGY

A Phonebook Cut and Rotated

Take a phonebook, cut it at a random page, and swap the two halves so it now starts mid-alphabet, wraps around, and ends just before where it used to start. It looks scrambled, but it's still two sorted halves glued together at the cut. Looking at the middle page, you can always tell which of the two halves you're straddling is internally sorted (compare the first visible page to the middle page), and that's enough to throw away half the phonebook again — exactly like ordinary binary search.

THE THEORY

Modified Binary Search extends the 'cut the search space in half every step' idea beyond plain sorted arrays — rotated sorted arrays, arrays of unknown length, finding a peak element, searching a 2D sorted matrix. The core discipline never changes: look at the middle, use some comparison to prove one half is fully sorted, fully disqualified, or fully irrelevant, and recurse into the other half. The variation is entirely in what that comparison checks.

- Compute $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$ (avoids overflow versus $(\text{left} + \text{right}) / 2$).
- If $\text{arr}[\text{mid}]$ is the target, return it immediately.
- Determine which half — left-to-mid or mid-to-right — is internally sorted by comparing $\text{arr}[\text{left}]$ to $\text{arr}[\text{mid}]$.
- Check whether the target could live in that sorted half's range; if yes recurse/loop there, if no go to the other half.

TIME COMPLEXITY

$O(\log n)$ — the search space halves every iteration

SPACE COMPLEXITY

$O(1)$ iterative; $O(\log n)$ if written recursively

WORKED EXAMPLE

Search in Rotated Sorted Array

You're given a sorted array that has been rotated at an unknown pivot. Given a target value, return its index, or -1 if it isn't present, in $O(\log n)$ time.

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:          # left half is sorted
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:                               # right half is sorted
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

C++

SEARCH IN ROTATED SORTED ARRAY

```
#include <vector>
using namespace std;

int searchRotated(vector<int>& nums, int target) {
    int left = 0, right = (int)nums.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) {          // left half sorted
            if (nums[left] <= target && target < nums[mid]) right = mid - 1;
            else left = mid + 1;
        } else {                               // right half sorted
            if (nums[mid] < target && target <= nums[right]) left = mid + 1;
            else right = mid - 1;
        }
    }
    return -1;
}
```

COMMON PITFALL

Using `<=` versus `<` inconsistently when checking which half is sorted causes off-by-one exclusion of the pivot element itself. Trace the case where `left == mid` by hand — a single or two-element remaining range is where these bugs hide.

PRO TIP

Don't memorize eight different rotated-array templates. Memorize one sentence: 'one of the two halves around mid is always properly sorted — figure out which, then check if the target fits inside that sorted half's range.'



Subsets / Backtracking



CORE SKILL

Include or exclude, one decision at a time, undo the moment you're done exploring.



THE ANALOGY

Building a Shopping List, One Item at a Time

You're deciding, item by item down a shopping list, whether each item goes into your basket. For every item you make a choice — in or out — and immediately explore every possibility that follows before backtracking to try the opposite choice for the same item. The moment you're done exploring a branch, you physically take that item back out of the basket before moving on — the basket must always reflect exactly the choices still in progress, never a leftover from an abandoned path.

THE THEORY

Backtracking systematically explores a decision tree of choices — include/exclude, which candidate to place next, which cell to fill — building a partial solution and undoing (popping) the most recent choice the instant a branch is fully explored or proven invalid. This 'undo' step is what separates backtracking from plain brute-force enumeration: it lets you prune early instead of wastefully generating every combination and filtering afterward.

- Add the current partial solution (even if empty) to the results, if the problem wants every prefix/subset.
- Loop over the remaining candidates starting from the current position.
- Choose one: push it onto the path, recurse one level deeper starting just past this candidate.
- Un-choose it: pop it back off the path before trying the next candidate — this is the 'backtrack' step.

TIME COMPLEXITY

$O(2^n)$ for subsets, $O(n!)$ for permutations — inherent to enumerating that many combinations

SPACE COMPLEXITY

$O(n)$ recursion depth plus $O(2^n)$ or $O(n!)$ to store all results

WORKED EXAMPLE

Subsets II (With Duplicates)

Given an integer array that may contain duplicates, return all possible subsets (the power set) without any duplicate subset appearing twice.

PYTHON

SUBSETS II (WITH DUPLICATES)

```
def subsets_with_dup(nums):
    nums.sort()
    result = []
    path = []

    def backtrack(start):
        result.append(list(path))
        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue # skip duplicate siblings at this depth
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result
```

C++

SUBSETS II (WITH DUPLICATES)

```
#include <vector>
#include <algorithm>
using namespace std;

void backtrack(vector<int>& nums, int start, vector<int>& path, vector<vector<int>>& result) {
    result.push_back(path);
    for (int i = start; i < (int)nums.size(); i++) {
        if (i > start && nums[i] == nums[i - 1]) continue; // skip duplicate siblings
        path.push_back(nums[i]);
        backtrack(nums, i + 1, path, result);
        path.pop_back();
    }
}

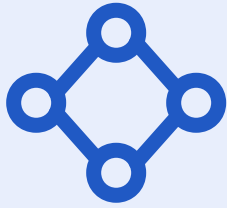
vector<vector<int>> subsetsWithDup(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> result;
    vector<int> path;
    backtrack(nums, 0, path, result);
    return result;
}
```

COMMON PITFALL

For the 'with duplicates' variant, forgetting to sort first and forgetting the `i > start` and `nums[i] == nums[i - 1]: continue` skip. Without both, the exact same subset gets generated multiple times.

PRO TIP

Draw the recursion tree for a 3-element input on paper before coding. If you can point to exactly where 'choose,' 'recurse,' and 'un-choose' happen on your drawing, the code basically writes itself.



PART IV

Graph Patterns

Two graph techniques that don't need a full BFS/DFS traversal to answer their question: one orders nodes under dependency constraints, the other tracks connectivity as edges are added one at a time.

14 · Topological Sort

15 · Union-Find (Disjoint Set)



Topological Sort


ADVANCED

Socks before shoes — find an order that respects every ‘before’ rule.



THE ANALOGY

Getting Dressed in the Right Order

You can't put on shoes before socks, or a jacket before a shirt. Some items have no prerequisite at all. A valid morning routine processes only items whose prerequisites are already satisfied, and every time you put something on, you ‘unlock’ whatever was waiting only on that item. If you ever get stuck holding clothes all waiting on each other with nothing free to put on next, your wardrobe rules contradict each other — there is no valid order.

THE THEORY

Topological Sort produces a linear ordering of a directed graph's nodes such that every edge $u \rightarrow v$ places u before v — but only when the graph is a DAG (Directed Acyclic Graph); a cycle makes no valid order possible. Kahn's Algorithm implements this via in-degree counting: track how many unsatisfied prerequisites each node has, start with every node at zero, and every time you ‘finish’ a node, decrement its neighbors' counts — a neighbor becomes eligible the instant its count hits zero. Finishing with fewer nodes processed than exist means a cycle blocked the rest.

- Build an adjacency list and an in-degree count array from every prerequisite edge.
- Seed a queue with every node that currently has in-degree zero.
- Pop a node, append it to the order, and decrement the in-degree of each of its neighbors.
- Any neighbor whose in-degree just hit zero gets pushed onto the queue. If the final order is shorter than the total node count, a cycle exists.

TIME COMPLEXITY

$O(V + E)$ — every node and edge is processed exactly once

SPACE COMPLEXITY

$O(V + E)$ for the graph plus the queue and in-degree array

WORKED EXAMPLE

Course Schedule II

There are n courses labeled 0 to n-1 and a list of prerequisite pairs. Return a valid order in which to take all courses, or an empty array if it's impossible.

PYTHON

COURSE SCHEDULE II

```
from collections import deque

def find_order(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1

    queue = deque([i for i in range(num_courses) if in_degree[i] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return order if len(order) == num_courses else []
```

C++

COURSE SCHEDULE II

```
#include <vector>
#include <queue>
using namespace std;

vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses);
    vector<int> inDegree(numCourses, 0);
    for (auto& p : prerequisites) {
        int course = p[0], prereq = p[1];
        graph[prereq].push_back(course);
        inDegree[course]++;
    }

    queue<int> q;
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) q.push(i);
    }

    vector<int> order;
    while (!q.empty()) {
        int node = q.front(); q.pop();
        order.push_back(node);
        for (int neighbor : graph[node]) {
            if (--inDegree[neighbor] == 0) q.push(neighbor);
        }
    }
}
```

```
return (int)order.size() == numCourses ? order : vector<int>();  
}
```

COMMON PITFALL

Building the edge direction backwards — pushing ‘prereq depends on course’ instead of ‘course depends on prereq’ silently inverts the whole ordering, sometimes still producing a plausible-looking but wrong answer.

PRO TIP

‘Prerequisite,’ ‘must happen before,’ ‘build order,’ and ‘dependency resolution’ are all this same idea in disguise — reach for in-degree counting and a queue.



Union-Find (Disjoint Set)



ADVANCED

Track 'same group' by pointing everyone at a shared leader.



THE ANALOGY

Merging Friend Groups at a Party

At a big party, you're tracking which friend groups have merged. Instead of remembering every single friendship, each person points toward whoever is currently considered their group's leader. When two groups meet and merge, one leader simply starts pointing at the other. To check whether two people are in the same group, follow each person's pointer-chain up to their respective leader and see if it's the same person — no need for the full history of who introduced whom.

THE THEORY

Union-Find (Disjoint Set Union) answers 'are these two things connected, transitively, through any chain of unions' in almost constant time per query — exactly what graph connectivity, cycle detection in undirected graphs, and Kruskal's MST need. Two optimizations make it fast: path compression (every find call flattens the chain so future lookups are instant) and union by rank/size (always attach the smaller tree under the bigger one's root, keeping chains short). Together they bring each operation to $O(\alpha(n))$ — the inverse Ackermann function, so close to constant it's treated as $O(1)$ in practice.

- Initialize every element as its own parent (n separate groups).
- $\text{find}(x)$: follow parent pointers up to the root, compressing the path (repointing nodes directly to the root) along the way.
- $\text{union}(x, y)$: find both roots; if they differ, attach the smaller-rank tree under the larger-rank one's root.
- Track a running count of distinct groups, decrementing it on every successful union.

TIME COMPLEXITY

$O(\alpha(n))$ per find/union — effectively constant for any realistic input size

SPACE COMPLEXITY

$O(n)$ for the parent and rank arrays

WORKED EXAMPLE

Number of Provinces

Given an $n \times n$ matrix where $isConnected[i][j] = 1$ means city i and city j are directly connected, return the total number of provinces (groups of directly or indirectly connected cities).

PYTHON

NUMBER OF PROVINCES

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
        self.count = n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        root_x, root_y = self.find(x), self.find(y)
        if root_x == root_y:
            return
        if self.rank[root_x] < self.rank[root_y]:
            root_x, root_y = root_y, root_x
        self.parent[root_y] = root_x
        if self.rank[root_x] == self.rank[root_y]:
            self.rank[root_x] += 1
        self.count -= 1

def find_circle_num(is_connected):
    n = len(is_connected)
    uf = UnionFind(n)
    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j] == 1:
                uf.union(i, j)
    return uf.count
```

C++

NUMBER OF PROVINCES

```
#include <vector>
using namespace std;

class UnionFind {
    vector<int> parent, rankArr;
public:
    int count;
    UnionFind(int n) : parent(n), rankArr(n, 0), count(n) {
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }
    void unite(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return;
    }
};
```

```

        if (rankArr[rootX] < rankArr[rootY]) swap(rootX, rootY);
        parent[rootY] = rootX;
        if (rankArr[rootX] == rankArr[rootY]) rankArr[rootX]++;
        count--;
    }
};

int findCircleNum(vector<vector<int>>& isConnected) {
    int n = isConnected.size();
    UnionFind uf(n);
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isConnected[i][j] == 1) uf.unite(i, j);
        }
    }
    return uf.count;
}

```

COMMON PITFALL

Implementing find without path compression. It still gives correct answers, but on a long chain of unions it degrades toward $O(n)$ per query instead of near- $O(1)$ — passes small tests, times out on large ones.

PRO TIP

‘Connected components,’ ‘provinces,’ ‘friend circles,’ or ‘redundant connection’ on an undirected graph — Union-Find is usually simpler to write correctly than a full BFS/DFS scan, especially when connections arrive one at a time.



PART V

Dynamic Programming

The pattern most candidates fear, because the code is short but the reasoning is not: define a state, define how smaller states combine into it, and let a table do the remembering so you never solve the same sub-problem twice.

16 · 0/1 Knapsack

17 · Unbounded Knapsack & String DP



BOSS LEVEL



0/1

Knapsack

Pack a suitcase where every item can go in exactly once — or not at all.

THE ANALOGY

Packing a Suitcase With a Weight Limit

You're packing one suitcase with a hard weight limit, and a pile of items, each with its own weight and value — but you only own one of each item. For every item you face one decision: does it earn its place given the room it costs, or is that room better spent elsewhere? The trap is that you can't decide item-by-item in isolation — taking a slightly-less-valuable-but-lighter item now might leave exactly enough room for two more items later. That's why you must remember, for every possible remaining capacity, the best value achievable — not just one running 'best so far.'

THE THEORY

0/1 Knapsack is the archetypal 'choose a subset under a capacity constraint, each item usable at most once' dynamic-programming pattern. The state is $dp[w]$ = the best total value achievable within a weight budget of w . For each item, you decide, for every capacity from large to small, whether including this item ($dp[w - \text{weight}] + \text{value}$) beats leaving it out ($dp[w]$ unchanged). Iterating the capacity loop backwards when using a 1D array is the whole trick — it guarantees you're reading last round's (pre-this-item) values, so you never accidentally reuse the same item twice.

- $dp[w]$ = best total value achievable within capacity w . Initialize $dp[0]$ appropriately for the exact problem variant.
- For every item, loop capacity w from the maximum down to the item's weight.
- Update: $dp[w] = \max(dp[w] \text{ (skip item)}, dp[w - \text{weight}] + \text{value} \text{ (take item)})$.
- After processing all items, $dp[\text{capacity}]$ holds the answer.

TIME COMPLEXITY

$O(n \cdot \text{capacity})$ — n items, each considered against every capacity value

SPACE COMPLEXITY

$O(\text{capacity})$ with the 1D rolling-array optimization

WORKED EXAMPLE

Partition Equal Subset Sum

Given an array of positive integers, determine whether it can be partitioned into two subsets with equal sum.

PYTHON

PARTITION EQUAL SUBSET SUM

```
def can_partition(nums):
    total = sum(nums)
    if total % 2 != 0:
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True
    for num in nums:
        for s in range(target, num - 1, -1): # backward: each item used once
            dp[s] = dp[s] or dp[s - num]
    return dp[target]
```

C++

PARTITION EQUAL SUBSET SUM

```
#include <vector>
#include <numeric>
using namespace std;

bool canPartition(vector<int>& nums) {
    int total = accumulate(nums.begin(), nums.end(), 0);
    if (total % 2 != 0) return false;
    int target = total / 2;
    vector<bool> dp(target + 1, false);
    dp[0] = true;
    for (int num : nums) {
        for (int s = target; s >= num; s--) { // backward: each item used once
            dp[s] = dp[s] || dp[s - num];
        }
    }
    return dp[target];
}
```

COMMON PITFALL

Looping the capacity dimension forward (low to high) instead of backward with a single 1D dp array. Forward iteration lets an item's own contribution feed back into itself later in the same pass, silently turning 0/1 Knapsack into Unbounded Knapsack.

PRO TIP

If you can rephrase the problem as 'each item used at most once, subject to a capacity,' it's 0/1 Knapsack — no matter how disguised (equal-subset-sum and target-sum-with-plus-minus-signs are this pattern wearing a costume).



BOSS LEVEL



Unbounded Knapsack & String DP

Same suitcase, but now every item has unlimited stock.

THE ANALOGY

Making Change With an Infinite Coin Jar

Unlike the suitcase with one of each item, imagine a coin jar with an unlimited supply of every denomination. Building a target amount, you're free to reuse the same coin denomination as many times as helps. This single difference — unlimited reuse — is why the capacity loop for this pattern runs forward instead of backward: you want an item's own contribution to be visible to itself later in the same pass, because that's exactly how 'reuse this coin as many times as needed' should behave.

THE THEORY

Unbounded Knapsack is 0/1 Knapsack's twin for problems where each item/coin can be reused unlimited times — coin change, rod cutting, unbounded fill problems. The recurrence looks almost identical, but looping capacity forward instead of backward is the entire difference, and it's exactly what makes reuse possible. A closely related family, DP on strings (Longest Common Subsequence, edit distance, longest palindromic subsequence), uses the same 'build a table of sub-problem answers, combine smaller solved sub-problems into bigger ones' philosophy, just indexed by two string positions instead of one capacity value.

- $dp[amount]$ = the best answer (min coins, ways to make change) for that exact amount. Initialize $dp[0]$ appropriately.
- For every coin/item, loop the target amount forward, from the coin's value up to the full target.
- Update: $dp[a] = \text{best}(dp[a], dp[a - \text{coin}] + 1)$ — $dp[a - \text{coin}]$ may already reflect this same coin being used earlier in this pass.
- After all coins are processed, $dp[target]$ holds the final answer.

TIME COMPLEXITY

$O(\text{amount} \cdot \text{number of coins})$

SPACE COMPLEXITY

$O(\text{amount})$ with the 1D rolling array

WORKED EXAMPLE

Coin Change (Minimum Coins)

Given an array of coin denominations and a target amount, return the fewest number of coins needed to make up that amount, or -1 if it's impossible.

PYTHON

COIN CHANGE (MINIMUM COINS)

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for a in range(coin, amount + 1):      # forward: reuse allowed
            dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1
```

C++

COIN CHANGE (MINIMUM COINS)

```
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;

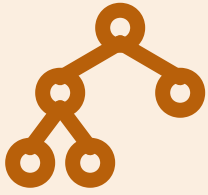
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int coin : coins) {
        for (int a = coin; a <= amount; a++) { // forward: reuse allowed
            if (dp[a - coin] != INT_MAX) {
                dp[a] = min(dp[a], dp[a - coin] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
```

COMMON PITFALL

Copy-pasting the 0/1 Knapsack backward loop out of habit. It compiles and even passes a few small test cases, then silently underestimates reuse-heavy answers because it never lets a coin help build an amount that then reuses that same coin again.

PRO TIP

One sentence separates the two Knapsacks forever: each item once → loop capacity backward; each item unlimited → loop capacity forward. If you remember only one fact from this chapter, remember that one.



PART VI

Strings & Stacks

A specialized tree for prefix-shaped string problems, and a stack that enforces its own ordering rule so aggressively that popping an element doubles as computing part of the answer.

18 · Trie (Prefix Tree)

19 · Monotonic Stack



Trie (Prefix Tree)



CORE SKILL

Every keystroke walks one more branch down a tree of shared prefixes.



THE ANALOGY

An Autocomplete Keyboard's Brain

Think about how a phone keyboard predicts words as you type. After 'c,' it already knows every word starting with 'c.' After 'ca,' the candidate list narrows to only words starting 'ca.' It isn't re-scanning a dictionary from scratch on every keystroke — it's walking one more step down a shared branching structure where every prefix in the entire dictionary that starts the same way literally shares the same path, splitting apart only where the words diverge.

THE THEORY

A Trie stores a set of strings so that any two strings sharing a prefix also share the tree path for that prefix, splitting into separate branches only where they diverge. Each node holds a map of 'next character → child node' plus a flag marking 'a complete word ends here.' This makes prefix queries (autocomplete, spell-check, word search on a grid) dramatically cheaper than repeatedly scanning a list of strings: checking whether any word starts with a given prefix costs only $O(\text{length of the prefix})$, independent of how many words are stored.

- **insert(word):** starting at the root, for each character, create a child node if it doesn't exist yet, then step into it.
- After the last character, mark that node as 'end of word.'
- **search(word) / startsWith(prefix):** walk the same way; if you can't find the next character's child, bail out immediately.
- search additionally requires the final node to be flagged 'end of word'; startsWith does not.

TIME COMPLEXITY

$O(L)$ per insert/search/prefix-check, L = length of the word or prefix

SPACE COMPLEXITY

$O(\text{total characters across all inserted words})$ in the worst case

WORKED EXAMPLE

Implement Trie (Prefix Tree)

Implement a Trie with insert, search (exact word match), and startsWith (prefix match) operations, each as efficient as possible.

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end = True

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find(prefix) is not None

    def _find(self, s):
        node = self.root
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

```

```

#include <string>
#include <unordered_map>
using namespace std;

class TrieNode {
public:
    unordered_map<char, TrieNode*> children;
    bool isEnd = false;
};

class Trie {
    TrieNode* root;
    TrieNode* find(const string& s) {
        TrieNode* node = root;
        for (char ch : s) {
            if (!node->children.count(ch)) return nullptr;
            node = node->children[ch];
        }
        return node;
    }
public:
    Trie() { root = new TrieNode(); }

```

```

void insert(const string& word) {
    TrieNode* node = root;
    for (char ch : word) {
        if (!node->children.count(ch)) node->children[ch] = new TrieNode();
        node = node->children[ch];
    }
    node->isEnd = true;
}

bool search(const string& word) {
    TrieNode* node = find(word);
    return node != nullptr && node->isEnd;
}

bool startsWith(const string& prefix) {
    return find(prefix) != nullptr;
}
};

```

COMMON PITFALL

Confusing search and startsWith. A prefix existing in the tree doesn't mean a complete word ends there — 'car' being a prefix of 'cart' doesn't make 'car' itself a stored word unless it was explicitly inserted.

PRO TIP

Whenever a problem is about prefixes specifically — autocomplete, longest common prefix among many strings, word search with a whole dictionary of targets — a Trie turns an $O(n \cdot L)$ repeated-scan approach into an $O(L)$ walk.



Monotonic Stack

A stack that only ever grows in one direction — and clears itself the instant that breaks.



THE ANALOGY

A Line of Bouncers Sorted by Height

Picture a line of bouncers, each one shorter than the one in front of them — that's the invariant they maintain. The moment a taller bouncer shows up wanting to join the back of the line, every shorter bouncer currently at the back has to step aside immediately (and conveniently, 'this taller person is the first taller person behind me' is exactly the fact each departing bouncer needed). Only once all the too-short bouncers have been dismissed does the tall newcomer take their place.

THE THEORY

A Monotonic Stack keeps its elements in strictly increasing or decreasing order at all times by popping off any element that would violate that order before pushing the new one — and the act of popping is itself the answer to 'what's the next greater/smaller element' for the thing being popped. This turns problems that look like they need $O(n^2)$ (comparing every element against every element ahead of it) into $O(n)$, because each element is pushed once and popped at most once across the entire algorithm's lifetime.

- Iterate through the array, maintaining a stack of indices (not values), kept in decreasing order of their values.
- For the current index i , while the stack isn't empty and the current value beats the value at the stack's top index, pop that index.
- Every popped index just found its answer: the current index i is its 'next greater element.'
- Push the current index onto the stack, and continue.

TIME COMPLEXITY

$O(n)$ total — every index is pushed once and popped at most once

SPACE COMPLEXITY

$O(n)$ worst case for the stack (e.g., a strictly decreasing input)

WORKED EXAMPLE

Daily Temperatures

Given a list of daily temperatures, return a list where each position holds the number of days you'd have to wait until a warmer temperature. If none exists, put 0.

PYTHON**DAILY TEMPERATURES**

```
def daily_temperatures(temperatures):
    n = len(temperatures)
    result = [0] * n
    stack = [] # stores indices, decreasing temperatures
    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            prev_index = stack.pop()
            result[prev_index] = i - prev_index
        stack.append(i)
    return result
```

C++**DAILY TEMPERATURES**

```
#include <vector>
#include <stack>
using namespace std;

vector<int> dailyTemperatures(vector<int>& temperatures) {
    int n = temperatures.size();
    vector<int> result(n, 0);
    stack<int> st; // stores indices
    for (int i = 0; i < n; i++) {
        while (!st.empty() && temperatures[st.top()] < temperatures[i]) {
            int prevIndex = st.top(); st.pop();
            result[prevIndex] = i - prevIndex;
        }
        st.push(i);
    }
    return result;
}
```

COMMON PITFALL

Storing values on the stack instead of indices. You need the index to compute ‘how many days until’ or ‘how far away’ — the value alone can’t tell you the distance back to where it came from.

PRO TIP

‘Next greater element,’ ‘next smaller element,’ ‘days until warmer,’ and ‘largest rectangle in a histogram’ are all the exact same monotonic-stack skeleton — only the comparison direction and what you do with a popped index changes.



PART VII

Optimization Toolkit

Two techniques that trade generality for speed: Greedy commits to a choice forever and never looks back, and Bit Manipulation squeezes hash-map-shaped problems into pure arithmetic on the bits themselves.

20 · Greedy Algorithms

21 · Bit Manipulation



Greedy Algorithms



ADVANCED

Make the locally best choice and trust it never needs revisiting.



THE ANALOGY

A Frog Crossing Lily Pads

A frog needs to cross a pond using lily pads of varying jump-distance, aiming for the fewest jumps possible. A greedy frog doesn't plan the whole crossing in advance — at each landing, it asks 'of every pad reachable from here, which one lets me reach the furthest pad after that?' and commits. It never reconsiders a jump once decided, because reaching further is provably at least as good as reaching less far, for any future the frog might face.

THE THEORY

Greedy algorithms build a solution by always taking the locally optimal choice at each step, and critically, never revisiting that choice. This only produces a correct global answer when the problem has the 'greedy-choice property': a locally best decision is always part of some globally optimal solution. That proof step is what separates Greedy from wishful thinking, and it's why Greedy is often harder to recognize than to code — the implementation is usually a short loop, but knowing why it's legal to never backtrack is the real skill.

- Identify the single quantity you're optimizing at each step (furthest reach, earliest finish time, cheapest next cost).
- At each position/step, scan only the locally available choices and pick the one that's best by that quantity.
- Commit to that choice permanently — no backtracking, no revisiting.
- Prove (at least informally, out loud in an interview) why a better later choice could never make an earlier greedy choice wrong.

TIME COMPLEXITY

$O(n)$ or $O(n \log n)$ if a sort is needed first — typically a single pass

SPACE COMPLEXITY

$O(1)$ beyond the input, in most greedy formulations

WORKED EXAMPLE

Jump Game II (Minimum Jumps to Reach the End)

Given an array where each element represents your maximum jump length from that position, return the minimum number of jumps required to reach the last index.

PYTHON**JUMP GAME II (MINIMUM JUMPS TO REACH THE END)**

```
def jump(nums):
    n = len(nums)
    jumps = 0
    current_end = 0
    farthest = 0
    for i in range(n - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = farthest
    return jumps
```

C++**JUMP GAME II (MINIMUM JUMPS TO REACH THE END)**

```
#include <vector>
#include <algorithm>
using namespace std;

int jump(vector<int>& nums) {
    int n = nums.size();
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < n - 1; i++) {
        farthest = max(farthest, i + nums[i]);
        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
```

COMMON PITFALL

Applying Greedy to a problem that doesn't actually have the greedy-choice property (classic trap: 0/1 Knapsack looks similar to Fractional Knapsack, but only the fractional version is safely greedy). Always sanity-check with a small counter-example before trusting a greedy idea in an interview.

PRO TIP

If you can't articulate why the locally best choice is always safe, you don't have a greedy solution yet — you have a guess. Interviewers will ask 'why does this work,' and 'it just does' doesn't pass.



ADVANCED



Bit Manipulation

Binary ballots where matching votes silently cancel out.

THE ANALOGY

Counting Votes That Cancel in Groups of Three

Imagine a ballot box where every candidate's name was submitted exactly three times, except one write-in candidate submitted only once. If any group of three identical ballots could physically vanish the instant the third match arrives, whatever's left standing at the end is the lone write-in. That's the intuition behind carrying two 'counter' bits per binary digit that automatically reset to zero every time they'd otherwise count to three — a tiny counting machine per bit position, built entirely out of XOR and AND.

THE THEORY

Bit Manipulation problems exploit the low-level properties of binary representation — XOR cancels identical bits, AND/OR isolate or combine bits, shifts multiply/divide by powers of two — to solve problems in $O(1)$ extra space that would otherwise need a hash map. The 'find the single number among duplicates' family is the classic showcase: plain XOR cancels pairs perfectly ($a \oplus a = 0$), and for 'every element three times except one,' you build a two-bit counter (ones, twos) per bit position that resets itself back to zero the instant it would count to three, using nothing but XOR and AND-NOT.

- For 'appears twice, find the one that appears once': XOR every element together — every pair cancels to 0, leaving only the lone value.
- For 'appears three times, find the one that appears once': maintain two accumulators, ones and twos, representing a per-bit counter mod 3.
- Update order matters: $\text{ones} = (\text{ones} \oplus \text{num}) \& \sim \text{twos}$, then $\text{twos} = (\text{twos} \oplus \text{num}) \& \sim \text{ones}$ — each line uses the other accumulator's freshly-updated state.
- After processing every element, ones holds the answer — the value that never got reset to zero.

TIME COMPLEXITY

$O(n)$ — one pass, $O(1)$ work per element

SPACE COMPLEXITY

$O(1)$ — a small fixed number of integer accumulators, no hash map needed

WORKED EXAMPLE

Single Number II

Given an array where every element appears exactly three times except for one element which appears exactly once, find that single element using only $O(1)$ extra space.

PYTHON

SINGLE NUMBER II

```
def single_number(nums):
    ones, twos = 0, 0
    for num in nums:
        ones = (ones ^ num) & ~twos
        twos = (twos ^ num) & ~ones
    return ones
```

C++

SINGLE NUMBER II

```
#include <vector>
using namespace std;

int singleNumber(vector<int>& nums) {
    int ones = 0, twos = 0;
    for (int num : nums) {
        ones = (ones ^ num) & ~twos;
        twos = (twos ^ num) & ~ones;
    }
    return ones;
}
```

COMMON PITFALL

Reaching for a hash map out of habit the moment a problem mentions counting occurrences. It works, but throws away the $O(1)$ -space constraint the problem is explicitly testing — and the bit-counter trick, once memorized, is barely more code than the hash map version.

PRO TIP

Don't try to derive the ones/twos update formulas from scratch under interview pressure — memorize the two-line pattern as a unit. It generalizes: 'every element k times except one' always reduces to a same-shaped multi-bit counter.

Quick-Reference Cheat Sheet

One line per pattern — for the morning of the interview, not for learning.

#	PATTERN	TIME	SPACE	REACH FOR IT WHEN...
01	Two Pointers	$O(n)$	$O(1)$ extra space	Walk toward the answer from both ends at once.
02	Sliding Window	$O(n)$	$O(k)$ for the window's tracking structure (k = alphabet size or window contents)	Grow and shrink a moving frame instead of re-scanning from scratch.
03	Fast & Slow Pointers	$O(n)$	$O(1)$	A tortoise and a hare always meet inside a loop.
04	Cyclic Sort	$O(n)$	$O(1)$	When numbers are their own array indices, sort by teleporting them home.
05	Merge Intervals	$O(n \log n)$	$O(n)$ for the output list	Sort by start time, then glue anything that overlaps.
06	In-place Reversal of a Linked List	$O(n)$	$O(1)$ iteratively; $O(n/k)$ recursion stack for the recursive k -group version	Flip pointers with only three hands, never derailing the train.
07	Tree BFS (Level Order)	$O(n)$	$O(w)$ where w is the widest level of the tree (worst case $O(n)$)	Explore a tree one generation at a time.
08	Tree DFS (Pre/In/Post-order)	$O(n)$	$O(h)$ recursion stack, h = tree height ($O(\log n)$ balanced, $O(n)$ worst case)	Commit to one path fully before you ever backtrack.
09	Two Heaps	$O(\log n)$ per insertion; $O(1)$ per median query	$O(n)$ to hold the full stream across both heaps	Keep the smaller half and the larger half perfectly balanced.
10	Top K Elements	$O(n \log k)$	$O(k)$ for the heap	Only k VIP seats — a newcomer must outrank someone to get in.
11	K-way Merge	$O(n \log k)$	$O(k)$ for the heap, plus $O(n)$ for the output	Merge k sorted lines by always pulling the smallest front.
12	Modified Binary Search	$O(\log n)$	$O(1)$ iterative; $O(\log n)$ if written recursively	Even a rotated or bounded array still has a 'sorted half' to search.
13	Subsets / Backtracking	$O(2^n)$ for subsets, $O(n!)$ for permutations	$O(n)$ recursion depth plus $O(2^n)$ or $O(n!)$ to store all results	Include or exclude, one decision at a time, undo the moment you're done exploring.
14	Topological Sort	$O(V + E)$		

			$O(V + E)$ for the graph plus the queue and in-degree array	Socks before shoes — find an order that respects every 'before' rule.
15	● Union-Find (Disjoint Set)	$O(\alpha(n))$ per find/union	$O(n)$ for the parent and rank arrays	Track 'same group' by pointing everyone at a shared leader.
16	● 0/1 Knapsack	$O(n \cdot \text{capacity})$	$O(\text{capacity})$ with the 1D rolling-array optimization	Pack a suitcase where every item can go in exactly once — or not at all.
17	● Unbounded Knapsack & String DP	$O(\text{amount} \cdot \text{number of coins})$	$O(\text{amount})$ with the 1D rolling array	Same suitcase, but now every item has unlimited stock.
18	● Trie (Prefix Tree)	$O(L)$ per insert/search/prefix-check, L = length of the word or prefix	$O(\text{total characters across all inserted words})$ in the worst case	Every keystroke walks one more branch down a tree of shared prefixes.
19	● Monotonic Stack	$O(n)$ total	$O(n)$ worst case for the stack (e.g., a strictly decreasing input)	A stack that only ever grows in one direction — and clears itself the instant that breaks.
20	● Greedy Algorithms	$O(n)$ or $O(n \log n)$ if a sort is needed first	$O(1)$ beyond the input, in most greedy formulations	Make the locally best choice and trust it never needs revisiting.
21	● Bit Manipulation	$O(n)$	$O(1)$	Binary ballots where matching votes silently cancel out.

21 Patterns Down.

Every pattern in this guide reduces to the same discipline: recognize the shape of the problem, not the problem itself. Two Sum and Partition Equal Subset Sum look nothing alike — until you notice one is Two Pointers and the other is 0/1 Knapsack wearing a costume.

Drill the recognition, and the code stops being the hard part.